ChaosFlow — Complete Project Guide

Resilient Big Data Pipeline Platform

A configuration-driven Big Data pipeline builder (PipeForge) combined with an automated reliability and fault-injection testing engine (ChaosLab).

Purpose of this document: to give every team member a deep, shared understanding of what we are building, why we are building it, how every part works, and how the parts fit together. Read it top to bottom once, then use the table of contents as a reference.

Table of Contents

1. The Big Picture 4
2. The Two Problems We Solve 4
3. Key Concepts You Must Understand First 5
4. What We Build vs. What We Reuse 6
5. System Architecture 7
6. End-to-End Data Flow 8
7. The Pipeline Configuration (YAML) 8
8. Configuration Validation Engine 10
9. Pipeline Model and Pipeline Generator 10
10. Plugin Architecture and Design Patterns 11
11. Data Sources 12
12. Processing Operations 12
13. Kafka Integration 13
14. Spark Integration 14
15. HDFS Integration 14
16. Airflow Integration 15
17. Pipeline Execution Manager 15
18. The Reliability Engine (ChaosLab) 16
19. Fault Types in Detail 17
20. Safety Mechanisms 18
21. Reliability Metrics and Formulas 19
22. Data Consistency Validation 19
23. Reliability Report and Reliability Score 20
24. Backend (FastAPI) 21
25. REST API Reference 21
26. Database Design 22
27. Authentication and Authorization 23
28. Frontend (React + TypeScript) 23
29. Monitoring: Prometheus and Grafana 24
30. Structured Logging 24
31. AI Module 24
32. Docker, Networking, and Configuration Management 25

33. Testing Strategy **26**

34. CI/CD with GitHub Actions **26**

35. Code Quality Standards **27**

36. Repository Structure **27**

37. Sample Datasets **27**

38. Final Demonstration Scenario **28**

39. Performance Benchmarking **28**

40. Extensibility **28**

41. Implementation Phases **29**

42. Rules of Engagement for the Team **29**

43. Known Limitations and Honest Caveats **30**

44. Acceptance Criteria **30**

45. Glossary **31**

46. FAQ **32**

1. The Big Picture

ChaosFlow is a platform with two halves:

Half	Name	Job
1	**PipeForge**	Lets a user *describe* a Big Data pipeline in a YAML file, and the platform builds, deploys, runs, and monitors it automatically.
2	**ChaosLab**	Lets a user *deliberately break* parts of that pipeline (stop Kafka, overload CPU, cut the network) in a controlled way, and measures how the pipeline survives.

One-sentence pitch:

> *Define your Big Data pipeline through configuration, deploy it automatically, monitor it continuously, intentionally test its resilience through controlled failures, measure its behavior, and optionally use AI to analyze reliability data.*

Analogy. Imagine a car factory.

- PipeForge is the *assembly line generator*: you hand it a blueprint (YAML) and it builds a working assembly line.
- ChaosLab is the *crash-test laboratory*: it deliberately stresses the line (cuts power, removes a worker, overheats a machine) and records exactly how fast it recovers and whether any cars were lost.

Why this is a strong graduation project:

- It touches many core CS and software-engineering topics: distributed systems, fault tolerance, messaging, big data processing, distributed storage, orchestration, DevOps, API design, databases, frontend, testing, and CI/CD.
- Its value is in the **custom software we write**, not in the tools we install. Kafka, Spark, and HDFS are components that *our platform controls*.
- It is fully runnable on a laptop using Docker Compose.

2. The Two Problems We Solve

Problem A — Repetitive Big Data pipeline development

Almost every Big Data project looks like this:

```
Data Source → Kafka → Spark → HDFS
```

Only these things change between projects:

- where the data comes from (CSV, JSON, API, logs, IoT sensors),
- the schema (column names and types),
- the cleaning/processing rules,
- where the results are stored.

The infrastructure (Kafka topics, Spark jobs, HDFS paths, monitoring) is configured again and again by hand. This is slow and error-prone.

Our answer: the developer writes a short YAML file. The platform validates it, generates everything needed, deploys it, and runs it.

Problem B — Nobody knows how the pipeline behaves when things break

After building a distributed pipeline, the real questions are:

- What happens if Kafka goes down for 20 seconds?
- If a Spark worker crashes mid-job, do we lose records?
- If the network between services is cut, how long until we recover?
- If CPU or memory is under pressure, how much does throughput drop?

Usually teams find out in production, during a real incident. This is expensive and dangerous. This discipline is called **Chaos Engineering** / **Reliability Engineering**.

Our answer: an automated engine that injects controlled, reversible failures, watches the system, and produces a measurable, repeatable report.

3. Key Concepts You Must Understand First

Every team member should be comfortable with these before touching the code.

3.1 Big Data and distributed systems

- **Big Data**: data too large or too fast for one machine to handle comfortably, so work is spread across many machines.
- **Distributed system**: several computers (or containers) cooperating to look like one system. Failures of individual parts are *normal*, not exceptional.
- **Fault tolerance**: the ability to keep working (or recover correctly) when parts fail.
- **Resilience**: how well a system absorbs failure and returns to normal.

3.2 Batch vs. streaming

- **Batch processing**: process a finished chunk of data (e.g., yesterday's file).
- **Streaming processing**: process records continuously as they arrive.
- ChaosFlow supports both: a CSV can be read as a batch and also published into Kafka as a stream.

3.3 Apache Kafka (messaging layer)

- A durable, distributed **log of messages**.
- **Producer** writes messages. **Consumer** reads messages.
- **Topic**: a named stream of messages (e.g., `sales-events`).
- **Partition**: a topic is split into partitions for parallelism. Order is guaranteed only within a partition.
- **Consumer group**: several consumers sharing the work of one topic; each partition is read by one consumer in the group.
- **Offset**: the position of a consumer in a partition. Committing offsets lets a consumer resume exactly where it stopped after a crash — this is central to our "no data loss" tests.
- **Consumer lag**: how far behind consumers are from the newest message. A rising lag is an early warning sign.
- **Replication factor**: how many copies of each partition exist on different brokers (limited in a single-broker local setup).

3.4 Apache Spark (processing engine)

- A distributed data-processing engine. Work is split into tasks run by **workers** (executors), coordinated by a **master/driver**.
- **DataFrame**: a table-like distributed data structure.
- **Transformation**: an operation producing a new DataFrame (filter, select, groupBy).

- **Structured Streaming**: Spark's API for continuous processing using the same DataFrame operations.
- **Checkpointing**: Spark saves its progress to storage so that after a crash it resumes instead of restarting. This enables fault-tolerant processing.
- **Partitioning**: how data is divided across tasks.

3.5 HDFS (distributed storage)

- Hadoop Distributed File System. Files are split into **blocks** and stored across **DataNodes**; the **NameNode** keeps the map of what is where.
- **Replication**: each block is stored multiple times so losing a node doesn't lose data.
- We use it as the final destination for processed data and verify the data exists after processing.

3.6 Apache Airflow (workflow orchestration)

- Schedules and runs **DAGs** (Directed Acyclic Graphs): ordered sets of tasks with dependencies.
- We use it for scheduled, repeatable pipeline workflows. It is *not* our main abstraction — our own platform is.

3.7 Observability

- **Metrics**: numbers over time (records/sec, errors, CPU).
- **Logs**: timestamped event records.
- **Prometheus** scrapes and stores metrics. **Grafana** draws dashboards from them.
- **Structured logging**: logs written as JSON so they can be searched and analyzed by machines.

3.8 Reliability vocabulary

- **Fault injection**: intentionally causing a failure.
- **Experiment**: one controlled test (target + fault + duration + expectations).
- **Baseline**: normal behavior measured *before* the fault, used for comparison.
- **Recovery time**: how long until the system works normally again after the fault.
- **Blast radius**: how much of the system a fault can affect. We keep it small and controlled.
- **Rollback / cleanup**: undoing the fault so the environment returns to normal.

3.9 Software engineering vocabulary

- **Plugin architecture**: new features are added as separate modules without editing the core.
- **Factory / Strategy / Registry / Dependency Injection**: design patterns explained in Section 10.
- **Clean architecture**: business logic is independent of web frameworks, databases, and external tools.
- **JWT**: JSON Web Token, a signed token proving who a user is.
- **RBAC**: Role-Based Access Control (ADMIN, ENGINEER, VIEWER).

4. What We Build vs. What We Reuse

This distinction is the heart of the project. Examiners will judge our engineering, not our ability to run `docker pull`.

Infrastructure we reuse (and orchestrate)

Kafka, Spark, HDFS, Airflow, PostgreSQL, Prometheus, Grafana, Docker.

Custom software we must write ourselves

1. Configuration parser
2. Pipeline definition model
3. Pipeline validation engine
4. Pipeline generator
5. Plugin architecture
6. Pipeline execution manager
7. Reliability testing engine
8. Fault injection mechanism
9. Metrics collection layer
10. Test result analyzer
11. Report generator
12. REST API
13. Web dashboard
14. Authentication/authorization foundation
15. AI analysis module

Rule of thumb: if a feature can be achieved just by editing a docker-compose file, it is not our contribution. Our contribution is the logic that *decides, generates, controls, measures, and explains*.

5. System Architecture

5.1 Logical layers

```
┌──────────────────────────────────────────────────────┐
│ Presentation Layer   React + TypeScript dashboard    │
├──────────────────────────────────────────────────────┤
│ API Layer            FastAPI REST API + Auth (JWT, RBAC)  │
├──────────────────────────────────────────────────────┤
│ Application Layer    Services: pipeline, experiment, report │
├──────────────────────────────────────────────────────┤
│ Domain / Core        Pipeline model, validators, plugins,  │
│                      reliability engine, analyzers, AI     │
├──────────────────────────────────────────────────────┤
│ Integration Layer    Kafka, Spark, HDFS, Airflow, Docker   │
│                      clients/adapters                      │
├──────────────────────────────────────────────────────┤
│ Infrastructure       Containers, network, volumes    │
└──────────────────────────────────────────────────────┘
```

5.2 Component diagram

```
User --> React Dashboard --> FastAPI Backend

FastAPI Backend --> PostgreSQL            (metadata, results)
FastAPI Backend --> Pipeline Engine       --> Airflow, Kafka, Spark, HDFS
FastAPI Backend --> Reliability Engine    --> Docker API --> Kafka, Spark, HDFS
FastAPI Backend --> AI Module

Data path:    Kafka --> Spark --> HDFS
Monitoring:   Backend /metrics --> Prometheus --> Grafana
```

5.3 Why this separation matters

- **Testability**: we can unit-test the validator or fault logic without running Kafka.

- **Replaceability**: swapping HDFS for MinIO means writing a new storage plugin, not rewriting the system.
- **Teamwork**: different members can work on different layers with clear interfaces.

6. End-to-End Data Flow

```
Source (CSV / JSON / API)
      |
      v
Ingestion Producer
      |
      v
Kafka Topic
      |
      v
Spark Structured Streaming Job
      |
      +--> HDFS (cleaned / aggregated data)
      |
      +--> Metrics --> Prometheus --> Grafana
```

Step by step:

1. **Source** — the configured source plugin reads records (e.g., rows of `sales.csv`).
2. **Ingestion producer** — each record is published to a Kafka topic (`sales-events`) with a **unique record ID** (needed later to detect loss).
3. **Kafka** — stores messages durably in partitions and holds consumer offsets.
4. **Spark** — a Structured Streaming job subscribes to the topic, applies the configured operations (remove nulls, filter, aggregate, etc.), and writes output.
5. **HDFS** — receives processed data (e.g., Parquet files) under the configured path.
6. **Validation** — the platform counts/reads records in HDFS and compares with what was sent.
7. **Metrics** — throughput, latency, errors, and counts flow to Prometheus and the backend.

Why Kafka sits in the middle: it **decouples** ingestion from processing and **buffers** data. If Spark is down for 20 seconds, Kafka keeps the messages, and Spark catches up on recovery. That buffering behavior is precisely what our reliability experiments verify.

7. The Pipeline Configuration (YAML)

The YAML file is the user's only interface to define a pipeline.

```yaml
pipeline:
  name: sales-pipeline
  version: "1.0"

source:
  type: csv
  path: /data/sales.csv

schema:
  - name: transaction_id
    type: string
  - name: customer_id
    type: string
  - name: amount
    type: double
  - name: timestamp
    type: timestamp

processing:
  operations:
```

```
    - type: remove_nulls
    - type: remove_duplicates
    - type: filter
      condition: "amount > 0"
    - type: aggregate
      group_by: customer_id
      operation: sum
      field: amount

streaming:
  enabled: true

messaging:
  broker: kafka
  topic: sales-events

processing_engine:
  type: spark

storage:
  type: hdfs
  path: /data/sales

monitoring:
  enabled: true
```

Section-by-section explanation

Section	Meaning
`pipeline`	Identity: unique `name` and a `version`. Versioning allows history and rollbacks (stored in `pipeline_versions`).
`source`	Where data comes from. `type` selects a **source plugin** (csv/json/api). Other keys are plugin-specific (path, URL, headers…).
`schema`	The expected columns and types. Enables validation of incoming data and typed Spark DataFrames. Supported types: string, integer, long, double, boolean, timestamp, date.
`processing.operations`	Ordered list of transformations. Order matters: operations run top to bottom. Each `type` maps to a **processor plugin**.
`streaming`	Whether the pipeline runs continuously (Structured Streaming) or as a batch.
`messaging`	Broker settings. Topic name, and optionally partitions, replication factor, consumer group.
`processing_engine`	Which engine executes the processing (initially Spark). Spark-specific tuning (partitions, executor memory) can go here.
`storage`	Where output goes. `type` selects a **storage plugin** (hdfs/local/minio), plus path/format/replication.
`monitoring`	Turns on metric emission and Prometheus integration.

> *The exact schema may be improved during implementation. Any change must be reflected in the Pydantic models, the validator, the sample YAMLs, and the docs together.*

Why YAML?

Human-readable, easy to version in Git, easy to diff, widely used in DevOps. The same file can be uploaded through the UI or the API.

8. Configuration Validation Engine

Why it exists: a wrong config must be caught *before* anything is deployed. Failing early is cheaper than failing inside Spark.

What is checked

- **Missing fields** (e.g., no `storage.path`)
- **Invalid types** (string where a number is expected)
- **Unsupported data source** (not registered as a plugin)
- **Unsupported processing operation**
- **Invalid schema** (duplicate column names, unknown types, empty schema)
- **Invalid storage configuration**
- **Invalid Kafka configuration** (bad topic name, partitions < 1, replication factor larger than allowed)
- **Invalid Spark configuration**
- **Cross-field rules**: e.g., `aggregate.group_by` must refer to a column that exists in the schema (or one created by an earlier operation); `filter.condition` must reference known columns.

How it works

1. YAML is parsed into a Python dictionary.
2. The dictionary is loaded into **Pydantic models** (structural and type validation).
3. A second layer of **semantic validators** applies cross-field and plugin-aware rules (asks the plugin registry whether a type exists, lets each plugin validate its own options).
4. All errors are **collected**, not just the first one, and returned in a consistent format.

Error format

```
{
  "valid": false,
  "errors": [
    {
      "field": "storage.path",
      "message": "Storage path is required"
    }
  ]
}
```

Good error messages are a feature: they should name the field, say what is wrong, and ideally suggest the fix.

9. Pipeline Model and Pipeline Generator

9.1 Pipeline definition model

An internal, typed representation of a pipeline (Pydantic/dataclasses), independent of YAML. It contains: metadata, source spec, schema, ordered operations, messaging spec, engine spec, storage spec, and monitoring spec.

Why: the rest of the platform works with this model, never with raw YAML. YAML is just one input format; JSON via the API produces the same model.

9.2 Pipeline generator

Job: turn the validated model into everything needed to run:

- the Kafka topic definition,
- the Spark job parameters (a **generic, parameterized Spark job** that interprets the pipeline model — we do *not* generate one hand-written script per pipeline),
- HDFS output directories,
- an Airflow DAG definition or DAG parameters,
- Prometheus metric labels for that pipeline.

Design decision: Spark jobs are **generic and configuration-driven**. The job receives the pipeline spec (JSON) as an argument, asks the plugin registry for each configured operation, and builds the DataFrame chain dynamically. This is what makes the platform truly "same engine, different dataset."

9.3 Pipeline lifecycle states

```
DRAFT → VALIDATED → DEPLOYED → RUNNING → STOPPED
                              ↘        ↘
                               FAILED   COMPLETED
```

State transitions are enforced by the service layer, and each is recorded (with timestamps) in the database and audit log.

10. Plugin Architecture and Design Patterns

Why it exists: hard-coding every source/processor/storage would force a core rewrite for each new feature. Plugins allow growth without touching the core.

10.1 Plugin categories

```
plugins/
  sources/       csv, json, api
  processors/    remove_nulls, remove_duplicates, filter, select, rename, cast, aggregate, transform
  storage/       hdfs, local, minio
```

(Fault types and AI models follow the same idea — see Section 40.)

10.2 How a plugin works

Each plugin:

1. Implements an **abstract base class / interface** (e.g., `SourcePlugin` with `validate_config()`, `read()`).
2. Declares its **type name** (e.g., `"csv"`).
3. **Registers itself** in a central registry (decorator or discovery on startup).
4. Provides its own configuration validation.

The core never imports a specific plugin; it asks the registry: "give me the plugin for type `csv`."

10.3 Design patterns used (and where)

Pattern	Where it appears	Why
Registry	Central plugin registry mapping type name → plugin class	Lookup without if/else chains
Factory	`PluginFactory.create("csv", config)` builds plugin instances	Core doesn't know concrete classes
Strategy	Each processor/source/storage/fault is an interchangeable strategy behind a common interface	Swap behavior at runtime by configuration
Dependency Injection	Services receive their dependencies (repositories, Kafka client, Docker client) via constructors/FastAPI `Depends`	Easy to mock in tests, low coupling
Template Method (optional)	Experiment lifecycle: fixed steps, fault-specific inject/cleanup	Consistent lifecycle across all fault types
Repository	Database access hidden behind repository classes	Business logic independent of SQLAlchemy

10.4 Adding a new plugin (conceptual recipe)

1. Create a folder/file under the right category.
2. Implement the interface.
3. Register it.
4. Add a unit test and a sample YAML.
5. No core files need editing.

11. Data Sources

Minimum required sources:

Source	Behavior
CSV	Reads rows from a CSV file, converts using the declared schema, emits records.
JSON	Reads JSON records (JSON lines or JSON array).
REST API	Pulls from an HTTP endpoint (URL, method, headers, pagination, polling interval). Needs timeout/retry handling.

Every source must:

- apply the schema,
- attach a **unique record ID** (existing key or generated UUID/hash) for loss detection,
- report errors per record without crashing the entire ingestion.

12. Processing Operations

Implemented as processor plugins using Spark DataFrame operations.

Operation	What it does	Typical config
`remove_nulls`	Drops rows with nulls (all columns or a chosen subset)	`columns: [...]` (optional)
`remove_duplicates`	Drops duplicate rows (by key columns or entire row)	`columns: [...]`

Operation	What it does	Typical config
`filter`	Keeps rows matching a condition	`condition: "amount > 0"`
`select`	Keeps only listed columns	`columns: [...]`
`rename`	Renames columns	`mapping: {old: new}`
`cast` (type conversion)	Converts a column's type	`column`, `to_type`
`aggregate`	Group + sum/avg/min/max/count	`group_by`, `operation`, `field`
`transform`	Basic derived columns (e.g., expression, uppercase, date extraction)	`column`, `expression`

Security note: `filter.condition` and `transform.expression` are user-provided expressions that reach Spark SQL. They must be validated (allowed columns, no dangerous constructs) and never executed as arbitrary code.

Important streaming caveat: aggregations and de-duplication in Structured Streaming require careful handling (watermarks, output modes, state). This must be documented and tested; streaming `aggregate` and `remove_duplicates` do not behave like their batch equivalents.

13. Kafka Integration

Responsibilities of our Kafka layer

- **Topic management**: create topics with configured partitions/replication; check if a topic exists.
- **Producer**: publish records (JSON-serialized) with the record ID as key or field.
- **Consumer**: read records for verification and for testing.
- **Consumer groups and offsets**: configure group IDs, inspect committed offsets, compute lag.
- **Health checks**: broker reachable, topic exists, lag within limits.
- **Readiness waiting**: other services wait until Kafka is *actually* ready, not just started.

Why it matters for reliability tests

When Kafka is stopped for 20 seconds:

- the producer should retry/buffer (producer settings: `acks`, `retries`, `delivery.timeout.ms`, idempotence),
- the consumer/Spark should reconnect,
- after recovery, offsets let processing resume without loss or (ideally) duplication.

Understanding delivery semantics is essential:

- **At-most-once** (may lose), **at-least-once** (may duplicate), **exactly-once** (hard, needs idempotence/transactions).

Our measured "data loss" and "duplicates" results are directly tied to these settings, and the report should state which semantics the configuration provides.

Kafka runs in **KRaft mode** (no ZooKeeper) to reduce components, unless a reason requires ZooKeeper.

14. Spark Integration

Responsibilities

- Run **real Spark jobs** inside Docker (no fake abstraction).
- Run a generic Structured Streaming job: Kafka → transformations → HDFS.
- Support batch mode for finite data.
- Configure partitioning, parallelism, and **checkpoint location** (stored on HDFS so it survives a Spark restart).
- Emit processing metrics (records processed, batch duration, errors).

Cluster layout

`spark-master` + one or more `spark-worker` containers. Jobs are submitted to `spark://spark-master:7077`.

Fault tolerance in Spark

- If a worker dies, the master reschedules its tasks elsewhere.
- If the driver/job dies, a restart with the same checkpoint resumes from the last committed offsets.
- Output sinks must be idempotent or transactional to avoid duplicates after retries — worth testing explicitly.

Required Kafka–Spark detail

Spark reads Kafka with the `kafka` source (`kafka.bootstrap.servers=kafka:9092`, `subscribe=<topic>`). The Spark-Kafka connector package version must match the Spark and Scala versions. Version mismatch is a classic source of errors — pin versions and document them.

15. HDFS Integration

Responsibilities

- **Write** processed output (Parquet recommended).
- **Read** processed data back for validation.
- **Check file/directory existence.**
- **Check health**: NameNode reachable, DataNodes live, safe mode off.
- **Replication configuration** (`dfs.replication`; with one DataNode, replication factor 1 — documented limitation).
- **Storage metrics**: used/available capacity, number of files, block counts where accessible (via WebHDFS or NameNode JMX).

Verification role

After processing, the platform confirms the expected output exists and counts the records. This is the **ground truth** for data-loss calculation.

Access options: WebHDFS REST API (simple from Python) or the Hadoop client. Choose one and use it consistently through a storage plugin interface.

16. Airflow Integration

Role

Airflow handles **scheduled workflow execution**. Our platform provides the higher-level abstraction; Airflow does not replace it.

Example DAG

```
Validate Configuration
        ↓
Prepare Source
        ↓
Start / Verify Kafka
        ↓
Run Spark Processing
        ↓
Validate Output
        ↓
Collect Metrics
```

How it connects

- Airflow tasks call our **backend API** (or share Python modules) rather than duplicating logic.
- A DAG can be generated/parameterized per pipeline by the generator.
- Run history in Airflow complements our own `pipeline_runs` table; our database remains the source of truth for the platform UI.

Keep it simple: do not rebuild Airflow's scheduler, UI, or retry system.

17. Pipeline Execution Manager

Why it exists: something must coordinate "deploy" and "run" across Kafka, Spark, HDFS, and Airflow.

Deploy

1. Load the validated pipeline version.
2. Verify infrastructure readiness (Kafka, Spark, HDFS health).
3. Create the Kafka topic if needed.
4. Create HDFS directories if needed.
5. Register/prepare the Airflow DAG and Spark job parameters.
6. Mark pipeline as `DEPLOYED`.

Run

1. Create a `pipeline_run` record.
2. Start the Spark streaming/batch job.
3. Start ingestion (source → Kafka).
4. Track state, counts, and errors; publish metrics.
5. On stop/finish, finalize the run record.

Principles

- Every step logs with the pipeline ID and run ID.
- Failures produce meaningful errors and move the run to `FAILED` without crashing the whole platform.

- Operations are idempotent where possible (deploying twice doesn't break things).

18. The Reliability Engine (ChaosLab)

One of the most important custom components. It is a **mini chaos-engineering framework**.

18.1 Experiment definition

```
experiment:
  name: kafka-failure-test

target:
  service: kafka

fault:
  type: container_stop
  duration: 20

validation:
  expected_data_loss: 0
  max_recovery_time: 30
```

Field	Meaning
`target.service`	Which allowed service/container to hit.
`fault.type`	Which fault plugin to use.
`fault.duration`	How long the fault lasts (seconds); must be ≤ the configured maximum.
`validation.expected_data_loss`	Acceptable lost records (usually 0).
`validation.max_recovery_time`	Pass/fail threshold for recovery time (seconds).

18.2 Experiment lifecycle

```
CREATE
  |
VALIDATE
  |
PREPARE
  |
BASELINE
  |
INJECT FAILURE
  |
MONITOR
  |
WAIT FOR RECOVERY
  |
VALIDATE DATA
  |
COLLECT METRICS
  |
ANALYZE
  |
CLEANUP
  |
GENERATE REPORT
```

What happens in each step:

Step	What happens
CREATE	Experiment saved with a unique **experiment ID**.

Step	What happens
VALIDATE	Target is in the allow-list; fault type exists; duration is within limits; pipeline is in a runnable state.
PREPARE	Ensure the pipeline is running; start a known dataset flow; record the expected record IDs/count.
BASELINE	Measure normal throughput, latency, CPU, memory for a short period. This is the reference.
INJECT FAILURE	Fault plugin applies the failure; timestamp recorded (T_fail). A timer guarantees the maximum duration.
MONITOR	Continuously sample service health, metrics, and logs during the fault.
WAIT FOR RECOVERY	After the fault ends (or is reverted), poll health until the service and pipeline are healthy again; timestamp recorded (T_recovered); give up after a timeout.
VALIDATE DATA	Compare expected vs. processed records (counts, IDs, optional checksums).
COLLECT METRICS	Aggregate recovery time, throughput, availability, error rate, resource usage.
ANALYZE	Compare against the baseline and the validation thresholds → PASS/FAIL, with reasons.
CLEANUP	Always runs (even after errors): revert faults, remove temporary files/processes, restore network.
GENERATE REPORT	Store a report and expose it via the API/dashboard.

18.3 Experiment states

`CREATED → VALIDATED → RUNNING → (COMPLETED | FAILED | CANCELLED | TIMED_OUT)` with a finer-grained `current_phase` field for the lifecycle step. Cancel requests trigger immediate cleanup.

18.4 Why cleanup is non-negotiable

A chaos tool that leaves the system broken is worse than useless. Cleanup is implemented with `try/finally` semantics, a watchdog timer, and a startup "orphan sweep" that reverts leftovers from crashed experiments.

19. Fault Types in Detail

Each fault is a **plugin** implementing: `validate()`, `inject()`, `revert()`, `is_active()`.

Fault	Idea	Typical mechanism	Reversal
Container Stop	Service disappears	Docker API: stop container	Start container
Container Restart	Service briefly dies and returns	Docker API: restart	Self-reverting; verify health
Network Disruption	Service unreachable but alive	Disconnect container from the project network (or `tc netem` packet loss inside the container)	Reconnect network / remove netem rule
CPU Stress	Resource contention	Run a stress process (e.g., `stress-ng`) inside the *target* container or a helper limited to that target for N seconds	Kill the stress process (auto-expires)
Memory Stress	Memory pressure	Allocate memory in a limited helper/process; rely on container memory limits to avoid harming the host	Terminate helper
Disk Pressure	Low disk space	Fill a *bounded* file inside the container or a dedicated, size-limited volume/tmpfs — never the host disk	Delete the file

Fault	Idea	Typical mechanism	Reversal
Service Delay	Slow service	`tc netem delay` on the container's interface (needs `NET_ADMIN` capability)	Remove netem rule

Notes:

- Some faults require extra Linux capabilities (`NET_ADMIN`) or tools inside containers. These are documented trade-offs.
- Docker Desktop on Windows runs containers in a Linux VM (WSL2); `tc` and stress tools work *inside* containers, so they are generally fine, but should be verified early.
- Every fault has a hard **maximum duration** and auto-reverts.

20. Safety Mechanisms

The reliability engine must **never blindly execute destructive commands on the host**.

Safeguard	Description
Target validation	The target must exist and be resolved by the engine, never by free-form user text.
Allow-list	Only listed services (e.g., kafka, spark-worker, namenode, datanode) may be targeted; platform-critical services (postgres, backend, the Docker socket proxy) are excluded by default.
Project scoping	Only containers carrying our Compose project label are eligible.
Experiment validation	Duration, fault parameters, and target compatibility are checked before running.
Timeouts	Global maximum experiment duration plus a per-phase timeout.
Automatic cleanup and rollback	Always executed, even on crash or cancel.
Experiment IDs	Every action is traceable.
Execution logs	Every command and state change is logged in structured form and attached to the experiment.
No shell string building	Use Docker SDK calls, not `os.system` with concatenated input (prevents command injection).
Role checks	Only ADMIN/ENGINEER can create/run experiments.

Docker socket warning

Giving a container access to the Docker socket is effectively **root-level control of the host**. Mitigations to document and apply:

- run the reliability component separately with the minimum necessary access,
- consider a **Docker socket proxy** that exposes only the needed API calls (list/stop/start/restart/exec),
- never expose it to the frontend or outside the Docker network.

21. Reliability Metrics and Formulas

Metric	Definition	Formula / How measured
Recovery time	Time from failure injection to confirmed recovery	`T_recovered − T_fail`, where recovery means service health OK *and* pipeline throughput returns to a threshold of baseline (e.g., ≥ 90%)
Data loss	Expected records not present in the output	`expected_count − unique_processed_count` (by record ID)
Duplicates	Records appearing more than once	`total_processed − unique_processed`
Throughput	Records processed per second	`records_processed / elapsed_seconds`, sampled per window; also *average* and *minimum during fault*
Throughput degradation	Drop vs. baseline	`(baseline − during_fault) / baseline × 100%`
Latency	Time from record creation/ingestion to storage	`t_processed − t_ingested` (needs timestamps); report mean/p95
Availability	Fraction of time the service was healthy	`healthy_samples / total_samples × 100%` or `uptime / total_time × 100%` over the experiment window
Error rate	Failed operations	`failed_operations / total_operations` and absolute counts
Resource usage	CPU and memory	Docker stats (CPU %, memory bytes/%) sampled at fixed intervals

Important: define *exactly* what "recovered" means and use the same definition in code, docs, and the report. Ambiguous definitions make results meaningless.

22. Data Consistency Validation

Goal: prove, with evidence, whether data was lost or corrupted.

Basic count check

```
Expected = 10,000
Processed = 10,000
→ Data Loss = 0

Expected = 10,000
Processed = 9,700
→ Data Loss = 300
```

Stronger check: unique record IDs

- Each record carries a unique ID from ingestion.
- After the experiment, the set of IDs in HDFS is compared with the set of IDs sent.
- Result: **missing IDs** (loss), **extra/duplicate IDs** (duplication).

Strongest check: checksums

- Compute a hash (e.g., SHA-256) over each record's content at ingestion and again at the output.
- A matching ID with a different checksum means **corruption**.

Caveat about transformations

If the pipeline filters or aggregates, "processed count" no longer equals "sent count." The validator must compute the **expected output** by applying the same logic to the sent records (or compare at a defined stage, such as "records consumed from Kafka" vs. "records written before aggregation"). For reliability experiments, use a pipeline configuration without count-changing operations or compute expectations explicitly.

23. Reliability Report and Reliability Score

Sample report

```
Experiment: Kafka Failure Test
Target: kafka
Failure Type: Container Stop
Failure Duration: 20 seconds
Recovery Time: 12.4 seconds
Expected Records: 10,000
Processed Records: 10,000
Data Loss: 0
Average Throughput: 1,250 records/sec
Availability: 99.8%
Result: PASS
```

Report contents

Experiment metadata, timeline with timestamps (inject, recover, validate), all metrics, comparison to baseline, validation thresholds and PASS/FAIL reasons, charts data, logs summary, and optional AI findings. Reports are stored historically and available in the dashboard (and exportable as JSON/HTML/PDF if time permits).

PASS / FAIL rule (example)

```
PASS if:
  data_loss <= expected_data_loss
  AND recovery_time <= max_recovery_time
  AND experiment completed without unhandled errors
else FAIL (with the failed criteria listed)
```

Reliability score (optional)

A transparent composite, never a mystery number. Example *proposal* (the team must finalize and document weights):

```
score = 100 × (
    0.35 × data_integrity_score
  + 0.25 × recovery_score
  + 0.20 × availability_score
  + 0.10 × throughput_retention_score
  + 0.10 × (1 − error_rate)
)
```

Each sub-score is normalized to 0–1 with a documented formula (e.g., `recovery_score = min(1, max_recovery_time / actual_recovery_time)`). The report must display the sub-scores so the score is auditable.

24. Backend (FastAPI)

Structure

```
backend/app/
  main.py          # app creation, router registration, middleware
  api/routes/      # thin HTTP layer
  core/            # config.py, security.py, logging setup
  models/          # SQLAlchemy ORM models
  schemas/         # Pydantic request/response models
  services/        # business logic orchestration
  pipeline/        # pipeline model, validator, generator, executor
  plugins/         # plugin base classes + registry
  reliability/     # experiment engine, faults, safety
  monitoring/      # metrics collection/export
  reports/         # report generation
  ai/              # anomaly detection, risk, recommendations
  database/        # session, repositories, migrations hooks
  utils/
backend/tests/
```

Clean architecture rules

- **Routes** only parse input, call a service, and return output.
- **Services** hold business logic and coordinate repositories and integrations.
- **Domain/core logic** (validators, fault logic, analyzers) does not import FastAPI or SQLAlchemy.
- **Integrations** (Kafka/Spark/HDFS/Docker clients) sit behind interfaces so they can be mocked.

Long-running work

Pipeline runs and experiments take time. They must **not** block HTTP requests: start them as background tasks (FastAPI background tasks for simple cases, or a worker/queue for robustness), return an ID immediately, and let the client poll or subscribe to status.

Error handling

- Central exception handlers return consistent JSON errors.
- All errors are logged with pipeline/experiment IDs.
- A failure in one pipeline/experiment must not crash the platform.

25. REST API Reference

Pipelines

```
POST   /api/pipelines
GET    /api/pipelines
GET    /api/pipelines/{id}
PUT    /api/pipelines/{id}
DELETE /api/pipelines/{id}
POST   /api/pipelines/{id}/validate
POST   /api/pipelines/{id}/deploy
POST   /api/pipelines/{id}/run
```

(Also needed for the UI: a stop endpoint such as `POST /api/pipelines/{id}/stop`.)

Experiments

```
POST   /api/experiments
GET    /api/experiments
GET    /api/experiments/{id}
POST   /api/experiments/{id}/run
POST   /api/experiments/{id}/cancel
```

Metrics

```
GET /api/metrics
GET /api/metrics/{pipeline_id}
```

Reports

```
GET /api/reports
GET /api/reports/{id}
```

Health / system

```
GET /health
GET /api/system/status
```

`/health` = is the backend alive. `/api/system/status` = status of Kafka, Spark, HDFS, Airflow, Postgres, etc.

Auth (needed)

Login, token refresh (optional), current-user endpoint, user management for ADMIN.

FastAPI auto-generates OpenAPI/Swagger docs at `/docs`.

26. Database Design

PostgreSQL + SQLAlchemy ORM + **Alembic** migrations.

Table	Purpose
`users`	Accounts: email/username, password hash, role, status
`pipelines`	Pipeline identity, owner, current status, current version pointer
`pipeline_versions`	Immutable stored config per version (YAML + parsed JSON), who/when
`pipeline_runs`	Each execution: start/end, status, counts, error info
`experiments`	Experiment definitions (target, fault, duration, thresholds)
`experiment_runs`	Each execution of an experiment: phase, timestamps, result
`fault_injections`	Every injection event: type, target, start/end, revert status, logs
`metrics`	Time-series-like measurements tied to pipeline/run/experiment (heavy time series live in Prometheus; we store summaries/samples needed for reports and AI)
`reports`	Generated reliability reports (structured JSON + summary + result)
`audit_logs`	Who did what, when (create/run/cancel/delete, login events)

```
USERS            1 --- N  PIPELINES
USERS            1 --- N  AUDIT_LOGS
PIPELINES        1 --- N  PIPELINE_VERSIONS
PIPELINES        1 --- N  PIPELINE_RUNS
PIPELINES        1 --- N  EXPERIMENTS
EXPERIMENTS      1 --- N  EXPERIMENT_RUNS
EXPERIMENT_RUNS  1 --- N  FAULT_INJECTIONS
EXPERIMENT_RUNS  1 --- N  METRICS
```

```
EXPERIMENT_RUNS   1 --- 1  REPORTS
```

Good practices: UUID primary keys, timestamps (`created_at`, `updated_at`), foreign keys with indexes, JSONB columns for flexible configs/metrics, and migrations for every schema change (never edit the database by hand).

27. Authentication and Authorization

Mechanisms

- **Password hashing** with a strong algorithm (bcrypt or argon2) — never store plain passwords.
- **JWT** access tokens (short expiry), signed with a secret from environment variables.
- **Role-based access control**:

Role	Can do
ADMIN	Everything, including user management and system settings
ENGINEER	Create/edit/deploy/run pipelines and run experiments
VIEWER	Read-only: dashboards, pipelines, reports

Permissions are enforced in the **API layer** with dependencies (e.g., `require_role(ENGINEER)`), and the frontend hides forbidden actions as a convenience only (the backend is the real gate).

Basic security checklist

Input validation everywhere, no secrets in Git, `.env` for configuration, restricted CORS, rate limiting on login (nice to have), audit logging of sensitive actions, and the experiment allow-list.

28. Frontend (React + TypeScript)

A professional dashboard-style single-page application.

Page	Contents
Login	Username/password, stores the JWT, redirects by role.
Dashboard	Active pipelines, running experiments, system health, recent failures, recovery-time trends, data loss, throughput.
Pipeline Management	List, create (form or YAML editor), upload YAML, edit, validate (shows field-level errors), deploy, run, stop.
Pipeline Details	Status; Kafka/Spark/HDFS status; processing statistics; run history; current config and versions.
Reliability Testing	Create experiment (target, fault, duration, thresholds), start, live monitor of the lifecycle phase and metrics, cancel.
Reports	Historical experiments; filters; per-report details; charts for recovery time, throughput, data loss, CPU, memory, error rate.

Frontend engineering notes:

- Typed API client generated or hand-written from the OpenAPI schema.
- State management with a data-fetching library (e.g., React Query) for polling/caching.
- Charts via a standard chart library.
- Route guards by authentication and role.
- Loading, empty, and error states on every page — a demo-quality UI needs them.

29. Monitoring: Prometheus and Grafana

Prometheus

- Scrapes a `/metrics` endpoint exposed by our backend (and optionally exporters for Kafka/Spark/HDFS/cAdvisor/node metrics).
- Example application metrics:

```
pipeline_records_processed_total
pipeline_processing_latency_seconds
pipeline_errors_total
pipeline_recovery_time_seconds
experiment_duration_seconds
```

Labels such as `pipeline_id`, `experiment_id`, `service` make slicing possible (watch out for **high-cardinality labels** — don't label with unbounded values).

Grafana

- Provisioned automatically (datasources and dashboards defined as files) so a fresh clone works without manual clicks.
- Suggested dashboards: Pipeline overview (throughput, latency, errors), Infrastructure (CPU/memory per container, Kafka lag), Reliability (recovery times, fault timeline annotations).
- During a demo, Grafana shows the **dip and recovery** visually — very persuasive.

Two paths for metrics

1. Prometheus → Grafana for live operations.
2. Backend collection → database → reports/dashboard/AI for experiment analysis.

30. Structured Logging

All services log JSON lines:

```
{
  "timestamp": "2026-01-01T12:00:00Z",
  "level": "INFO",
  "service": "reliability-engine",
  "pipeline_id": "pl-42",
  "experiment_id": "exp-123",
  "message": "Failure injection started",
  "error": null
}
```

Rules:

- Same field names everywhere.
- Include IDs on every relevant event.
- Include error type and stack trace on errors.
- Logs of an experiment are stored/attachable to the report and are an input to the AI module.
- Never log secrets or passwords.

31. AI Module

Optional. The platform must fully work with the AI module disabled.

Capabilities

Capability	What it does	Approach
Anomaly detection	Flags unusual CPU, memory, latency, throughput, error rate, recovery time	Scikit-learn: Isolation Forest, One-Class SVM, or simple statistical methods (z-score, IQR) trained on historical/baseline metrics
Failure-risk indication	Estimates whether the system is drifting toward an abnormal state	Probability/risk indicator from features such as rising lag, rising error rate, shrinking headroom — with clear stated limitations
Root-cause assistance	Suggests which services likely contributed	Correlate timelines of metrics, logs, service states, and experiment events; rank likely contributors
Recommendations	Explainable suggestions	Rule-based and evidence-linked text, e.g. "Consumer lag rose before failure; consider more consumers, review partitions, check processing latency"
RAG reliability assistant (planned extension)	Lets users ask questions about their own reports/logs/docs in natural language	Retrieval over stored reports, logs, and documentation, with answers grounded in cited sources

Honesty principles

- Never claim perfect prediction or guaranteed root cause.
- Present outputs as **risk levels / probabilities with confidence and caveats**.
- Every recommendation shows its **evidence** (the metric or log line that triggered it).
- Small datasets limit model quality: document how much history is needed and what happens with too little.

Pipeline

```
metrics + logs + experiment history → feature extraction → model/rules → findings → stored with report → shown in da
```

32. Docker, Networking, and Configuration Management

Services in Docker Compose

```
frontend, backend, postgres,
kafka (KRaft or with zookeeper),
spark-master, spark-worker,
namenode, datanode,
airflow (webserver/scheduler + metadata DB),
prometheus, grafana
(+ optional exporters, docker-socket-proxy)
```

Rules

- **Dedicated Docker network**; services talk by name: `kafka:9092`, `postgres:5432`, `spark-master:7077`, `namenode:9000`.
- **Persistent volumes** for Postgres, Kafka, HDFS, Grafana, Prometheus.
- **Health checks** for every service.
- **Real readiness**: `depends_on: condition: service_healthy` plus application-level waiting (e.g., "can I actually list Kafka topics?") because "container started" ≠ "service ready."
- **Resource limits** (CPU/memory) so a laptop is not overwhelmed — the full stack is heavy; budget RAM (a realistic minimum is around 12–16 GB of RAM available to Docker for the whole stack, to be

verified).

- **Compose profiles** so developers can start only what they need (e.g., core, full, monitoring).

Configuration management

- `.env.example` is committed; `.env` is **never** committed.
- Safe development defaults only; no real secrets in the repository.
- All environment-specific values (hosts, ports, secrets, limits) come from environment variables read in `core/config.py` (Pydantic Settings).

Windows/WSL2 notes

Docker Desktop with WSL2 backend; keep the repo inside the WSL2 filesystem for performance when possible; line endings (use LF) matter for shell scripts and container entrypoints; allocate enough memory in `.wslconfig`.

33. Testing Strategy

Testing is mandatory.

Level	Scope	Examples
Unit	Single components, no external services	Config parser, validators, plugin registry/factory, pipeline generator, reliability state machine & safety checks, metric calculations, report generation
Integration	Our code with real services	Kafka produce/consume; Spark job on real data; PostgreSQL repositories + migrations; HDFS write/read/exists
End-to-end	Whole workflow	Upload YAML → validate → deploy → send data → process → store → run failure experiment → recover → verify data → generate report
Frontend	UI behavior	Component tests and a few critical flows
Safety tests	The dangerous parts	Allow-list rejects forbidden targets; max duration enforced; cleanup runs after forced errors

Principles:

- Test the **reliability engine's safety** more thoroughly than anything else.
- Tests must be able to run in CI.
- Never mark a reliability test as passing unless it was actually executed against real services.

34. CI/CD with GitHub Actions

```
Push → Lint → Unit Tests → Integration Tests → Build Docker Images → Security Checks → Test Report
```

- Lint: Ruff, Black (check mode), ESLint, Prettier (check).
- Type checks: MyPy where practical.
- Unit tests: fast, no Docker services.
- Integration tests: start required services with Compose inside the runner (resource-heavy; run a reduced stack).
- Security checks: dependency auditing (e.g., pip-audit, npm audit) and image scanning (e.g., Trivy).
- Artifacts: test and coverage reports.
- **No automatic production deployment** — this is a local/academic project.

35. Code Quality Standards

- **Python**: Black, Ruff, MyPy (practical), Pytest.
- **Frontend**: ESLint, Prettier, strict TypeScript.
- Clear naming, docstrings for important components, minimal unnecessary comments.
- Principles: SOLID, DRY, KISS (where appropriate), separation of concerns, dependency inversion, high cohesion, low coupling, extensibility, testability, observability.
- Avoid over-engineering: build the simplest design that satisfies the requirement and keeps extension points where the requirements say they are needed.
- Git workflow: feature branches, pull requests with review, meaningful commits, no secrets or large data in Git.

36. Repository Structure

```
resilient-big-data-platform/
├── backend/                # FastAPI app, tests, Dockerfile
├── frontend/               # React + TypeScript app
├── pipeline-engine/        # config → pipeline generation/execution logic (may live inside backend)
├── reliability-engine/     # experiments, faults, safety (may live inside backend)
├── spark-jobs/             # generic Spark batch/streaming jobs
├── airflow/dags/           # DAG definitions
├── infrastructure/         # kafka, spark, hdfs, prometheus, grafana configs
├── plugins/                # sources, processors, storage (and more)
├── sample-data/            # sales, IoT, network/log datasets
├── experiments/            # example experiment YAMLs
├── scripts/                # helper scripts (wait-for, seed, demo)
├── tests/                  # cross-component integration and e2e tests
├── docs/                   # architecture, guides, diagrams
├── docker-compose.yml
├── .env.example
├── .gitignore
├── README.md
└── LICENSE
```

The structure may be improved if a better architecture is justified. A key team decision: whether `pipeline-engine` and `reliability-engine` are separate Python packages/services or modules inside the backend. Separate packages give clearer boundaries; modules inside the backend are simpler to run.

37. Sample Datasets

Dataset	Fields
Sales	`transaction_id`, `customer_id`, `product_id`, `amount`, `timestamp`
IoT	`device_id`, `temperature`, `humidity`, `timestamp`
Network/Log	e.g., `timestamp`, `src_ip`, `dst_ip`, `src_port`, `dst_port`, `protocol`, `bytes`, `status`

Purpose: prove that **the same engine** handles different domains using only different YAML files. Include datasets in multiple sizes (small for tests, larger for performance runs) and a **data generator script** that can create arbitrary amounts of realistic records with unique IDs, plus a few deliberately dirty rows (nulls, duplicates, negative amounts) to demonstrate the processing operations.

38. Final Demonstration Scenario

Step	Action
1	Open the dashboard.
2	Create a Sales pipeline from YAML.
3	Validate the configuration (also show a deliberately broken config with clear errors).
4	Deploy the pipeline.
5	Generate/send sample data.
6	Show Kafka receiving messages.
7	Show Spark processing them.
8	Show HDFS storing the results.
9	Show metrics live in Grafana.
10	Start a reliability experiment: Target = Kafka, Fault = Container Stop, Duration = 20 s.
11	Show the Kafka failure happening.
12	Show pipeline behavior during the failure (throughput dip, producer retries).
13	Show automatic recovery.
14	Show Recovery Time, Data Loss, Throughput, Availability.
15	Show the generated reliability report.
16	Show the AI analysis.

Demo tips: rehearse repeatedly; pre-warm the stack (cold starts are slow); keep a recorded backup of the demo; prepare a second dataset (IoT) and deploy it live to show the platform is truly generic.

39. Performance Benchmarking

Benchmark and **store results** for:

- ingestion throughput (records/sec into Kafka),
- processing throughput (records/sec through Spark),
- API response times,
- recovery time per fault type,
- data validation time.

Method: fixed dataset sizes, repeated runs (e.g., 3–5), report mean and variance, record machine specs (CPU, RAM, Docker limits). Honest numbers from a laptop are fine — document the environment.

40. Extensibility

The system must let future developers add the following **without rewriting the core**:

Extension	How
New data source	New source plugin
New processing operation	New processor plugin
New storage backend	New storage plugin

Extension	How
New fault type	New fault plugin (inject/revert/validate)
New monitoring metric	Register a metric in the metrics layer
New AI model	New analyzer implementing the AI interface

Each extension point has: an abstract interface, a registry, a documented example, and tests. If adding something requires editing core files, the design needs fixing.

41. Implementation Phases

Phase	Content
1	Repository, architecture, Docker base
2	PostgreSQL + FastAPI
3	Configuration system
4	Pipeline engine
5	Kafka
6	Spark
7	HDFS
8	Airflow
9	Reliability engine
10	Metrics + Prometheus
11	Grafana
12	React dashboard
13	Testing
14	CI/CD
15	AI module
16	Documentation and final integration

Critical rule after each phase: build containers → start services → health checks → run tests → verify functionality → fix errors → only then continue. Never wait until the end to discover that components cannot communicate.

> *Note: the phase numbering above follows the original master prompt. Where the team's actual progress groups work differently (for example, merging configuration and plugin work), keep the same verification rule at every boundary.*

42. Rules of Engagement for the Team

1. **No fakes.** No simulated Kafka/Spark/HDFS, no fabricated API responses, no claiming tests passed without running them.
2. **No TODO placeholders in core features.** If something is not done, it is tracked explicitly as not done.

3. **Everything reproducible.** A new developer clones, copies `.env.example` to `.env`, runs Compose, and gets a working system.
4. **Every feature has tests.** Especially safety logic.
5. **Document limitations honestly.** If local Docker prevents something (e.g., real multi-broker replication), say so and provide the working alternative.
6. **Small PRs, early integration.** Integrate continuously.
7. **Shared definitions.** Metric definitions, state names, and error formats are written once in docs and used everywhere.
8. **Security by default.** Secrets in environment, least privilege for the Docker socket, validated inputs.

43. Known Limitations and Honest Caveats

- **Resource heaviness:** Kafka + Spark + HDFS + Airflow + Prometheus + Grafana on one laptop is demanding. Use resource limits and Compose profiles; consider a lighter dev profile.
- **Single-node realism:** a local setup has typically one broker and one or two DataNodes, so replication-related behavior is limited. Document what the experiments can and cannot show.
- **Streaming semantics:** exactly-once end-to-end requires deliberate design (idempotent sinks, checkpoints, transactional producers). Be explicit about what guarantee the project provides.
- **Fault realism:** container-level faults approximate real failures; they are not identical to hardware or datacenter failures.
- **Docker socket risk:** needed by the reliability engine; mitigated, not eliminated.
- **AI limits:** limited training data from a student environment means AI outputs are indicative, not authoritative.
- **Windows specifics:** WSL2 memory limits, file-system performance, and line endings can cause confusing errors.

Stating these openly is a strength in a graduation defense.

44. Acceptance Criteria

The project is complete only if all of the following work:

Area	Requirement
Pipeline	A user can create a YAML configuration
Validation	The configuration is validated with useful errors
Deployment	The system deploys the required services/resources
Ingestion	Data enters Kafka
Processing	Spark processes the data
Storage	Results are stored in HDFS
Monitoring	Metrics are collected
Reliability	A failure can be injected safely
Recovery	The system recovers where the architecture supports it
Measurement	Recovery time and data consistency are measured
Reporting	A reliability report is generated and stored

Area	Requirement
Dashboard	Pipeline and reliability information is viewable
AI	The optional module analyzes historical metrics
Testing	Automated tests exist and pass
Documentation	Another developer can clone and run it using the README

45. Glossary

Term	Meaning
Availability	Percentage of time a service is healthy and usable
Baseline	Normal measured behavior before a fault
Blast radius	Scope of impact of a fault
Broker	A Kafka server
Chaos engineering	Practice of deliberately injecting failures to learn how a system behaves
Checkpoint	Saved processing progress enabling resume after failure
Consumer group	Set of consumers sharing a topic's partitions
Consumer lag	Number of messages a consumer is behind
DAG	Directed Acyclic Graph; Airflow workflow definition
DataNode / NameNode	HDFS storage node / HDFS metadata server
DataFrame	Table-like distributed dataset in Spark
Fault injection	Intentionally causing a failure
Idempotent	Repeating an operation gives the same result as doing it once
JWT	Signed token for authentication
KRaft	Kafka's built-in consensus mode replacing ZooKeeper
Offset	A consumer's position in a Kafka partition
Partition	Subdivision of a Kafka topic enabling parallelism
Plugin	Self-contained extension registered with the core
RBAC	Role-based access control
Recovery time	Time from fault injection to confirmed recovery
Replication factor	Number of copies of data kept
Rollback	Reverting changes after a fault/experiment
SRE	Site Reliability Engineer
Structured Streaming	Spark's continuous processing API
Throughput	Records processed per unit of time
Topic	Named Kafka stream of messages
Watermark	Spark mechanism to bound state for late data in streaming

46. FAQ

Is this just a Docker Compose file? No. Compose is only the environment. The project is the custom platform: parser, validator, generator, plugin system, execution manager, reliability engine, analyzers, API, dashboard, and AI module.

Why Kafka between the source and Spark? It decouples and buffers. It also provides offsets, which make recovery and data-loss measurement meaningful.

Why a generic Spark job instead of generating code per pipeline? A single, tested, configuration-driven job is safer, easier to test, and demonstrates the "same engine, different dataset" idea cleanly.

What exactly counts as "recovery"? It must be defined once and used everywhere: the target service is healthy again *and* pipeline throughput is back above a threshold of baseline (for example 90%). Document the chosen definition.

What if an experiment crashes halfway? Cleanup runs regardless (try/finally, watchdog, and orphan sweep at startup). The experiment is marked FAILED with logs.

Can the platform work without AI? Yes. AI is a layer on top of stored metrics and reports.

Why does the Docker socket matter? Access to it is powerful. Use the minimum privileges (socket proxy), allow-lists, project label scoping, and document the risk.

How do we prove there was no data loss? Unique IDs at ingestion, ID/count comparison at the output, and optional checksums for corruption.

How do we show the platform is generic? Deploy a second dataset (IoT or network logs) with only a different YAML during the demo.

What is the single most important thing to get right? A clean separation between our custom logic and the infrastructure, with a safe, honest, measurable reliability engine at the center.